```
import { ChatOpenAI } from "@langchain/openai";
import "dotenv/config";
const apiKey = process.env.OPENAI_API_KEY;

const toUpperCase = (input) => {
  return {
    uppercased: input.text.toUpperCase(),
  };
};

const vowelCountFunction = (input) => {
  const vowels = input.uppercased.match(/[AEIOU]/gi);
  return {
    vowelCount: vowels ? vowels.length : 0,
  };
};
const model = new ChatOpenAI({ openAIApiKey: apiKey,
model: "gpt-4o" });
const prompt = ChatPromptTemplate.fromTemplate("Show the number {vowelCount}
two times.");
const chain = RunnableLambda.from(toUpperCase).
  .pipe(RunnableLambda.from(vowelCountFunction))
  .pipe(prompt)
  .pipe(model)
  .pipe(new StringOutputParser());

await chain.invoke({text: "hello world"})
.then((output) => {
    console.log(output);
});
```

**Takes an input object and converts the "text" property to uppercase**

**Counts the number of vowels in the uppercase text**

**Initializes the ChatOpenAI model with the API key and specifies the model to use**

**Creates a prompt template that will be used to format the input for the language model**

**Creates a chain of operations**

**Invokes the chain with the input "hello world" and logs the output**

We start by defining two functions: `toUpperCase` to convert text to uppercase and `vowelCountFunction` to count vowels in the uppercase text. It's crucial to pay attention to the order of these pipe events, as they determine the flow of data processing. The chain first converts the input to uppercase, then counts the vowels, applies a prompt template with the vowel count, passes it to the `ChatOpenAI` model, and finally parses the output as a string. This sequence ensures that each step builds upon the results of the previous one to produce the final output. The chain is then invoked with the input `"hello world"`, and the result is logged to the console.

With the prompt now set to `"Show the number {vowelCount} two times,"` the language model will generate a more conversational response. For example, when running `simple-chain.openai.js`, you might see output similar to this:

```
% node simple-chain.openai.js
3 3
```

Alternatively, you can achieve the same functionality using `RunnableSequence.from`. This method allows you to create a sequence of runnables that are executed in order, which can be particularly useful for operations that are independent yet need to be